

TRABAJO PRÁCTICO UNIDAD 1 — PARTE 2

Repositorios de Software — Metodología de Sistemas II — TUP 2026

Centro Educativo "Educar para Transformar"

Equipo de trabajo: Fabián Trangoni / Bruno Mazza

Fecha: 25 de agosto de 2026

1. Terminología de Repositorios

Traducción al castellano de los principales conceptos de Git y sistemas de control de versiones:

Concepto	Significado
Branch	Rama: permite desarrollar una funcionalidad o corregir un error sin modificar el proyecto principal.
Commit	Confirmación: un cambio guardado, generalmente acompañado de un mensaje breve que describe lo realizado.
Pull	Traer/Actualizar: descarga al equipo local los últimos cambios del repositorio remoto.
Pull Request	Solicitud de incorporación: pide al equipo revisar los cambios propuestos e incorporarlos al proyecto principal.
Push	Subir/Enviar: envía los cambios locales al repositorio remoto para que el resto del equipo pueda usarlos.
Merge	Fusionar: combina los cambios de una rama con otra (habitualmente con la rama principal).
Clone	Clonar: crea una copia completa de un repositorio remoto en el equipo local para trabajar en él.
Fork	Bifurcar: copia personal del repositorio de otra persona, que permite experimentar sin afectar el original.
Tag	Etiqueta: marca un punto específico del historial (por ejemplo, una versión o release) con un nombre identificable.
Pipeline	Canalización: secuencia de pasos automatizados (compilar, testear, desplegar) que se ejecuta al integrar cambios.
CI/DI (CI/CD)	Integración Continua / Entrega Continua: prácticas que automatizan la integración, prueba y entrega frecuente del código.

2. Verdadero o Falso

Se analizaron las cinco afirmaciones contrastándolas con los gráficos de arquitectura provistos. Las cinco resultaron verdaderas:

N°	Afirmación	V/F	Justificación
1	GitHub, GitLab y Bitbucket permiten usar repositorios Git en entornos colaborativos, con revisión de código, gestión de incidencias, administración de proyectos, automatización, integración y entrega continua.	V	Estas plataformas usan Git como base y agregan servicios de colaboración (Issues, Pull Requests, CI/CD).
2	Git y GitHub no son lo mismo: Git es el sistema de control de versiones distribuido, y GitHub es una plataforma que utiliza Git y agrega servicios de colaboración y gestión.	V	Git es la herramienta; GitHub, GitLab y Bitbucket son plataformas que la alojan y potencian con funcionalidades extra.
3	En un sistema distribuido, cada clon constituye una copia completa del repositorio y su historial, aportando capacidad de recuperación ante fallos.	V	Al tener cada integrante el historial completo, cualquier clon puede restaurar el repositorio central si este se pierde.
4	En arquitectura centralizada, el servidor está conectado directamente al puesto de trabajo de cada programador, quienes actualizan (update) o modifican (commit) directamente sobre el repositorio.	V	Coincide con el primer gráfico: no existe repositorio local intermedio, todo ocurre contra el servidor central.
5	En arquitectura distribuida, los programadores	V	Coincide con el segundo gráfico: cada repositorio local

	actualizan sus repositorios locales con 'pull' y persisten cambios en el repositorio principal con 'push' desde su repositorio local.		sincroniza con el servidor vía push/pull, y con la copia de trabajo vía commit/update.
--	---	--	--

3. Interpretación de Gráficos de Arquitectura

Gráfico 1

Arquitectura	Explicación de su funcionamiento
Centralizada	Existe un único servidor central que aloja el repositorio. Cada puesto de trabajo se conecta directamente a él: 'update' baja los cambios existentes y 'commit' sube las modificaciones. No hay una copia local del historial completo, por lo que la disponibilidad del proyecto depende enteramente del servidor central.

Gráfico 2

Arquitectura	Explicación de su funcionamiento
Distribuida	Cada programador posee una copia completa del repositorio (con su historial) en su propio equipo. Sincroniza con el servidor central mediante 'pull' (bajar cambios) y 'push' (subir cambios); dentro de su equipo, usa 'commit' y 'update' entre su copia de trabajo y su repositorio local. Al existir múltiples copias completas, el sistema gana tolerancia a fallos: cualquier clon puede restaurar el repositorio central.

4. Portafolio Digital

Se seleccionó GitHub como repositorio de datos para construir el portafolio digital del equipo, aprovechando el mismo repositorio ya creado para el proyecto. La estructura propuesta es la siguiente:

- /01-Requerimientos → Especificación de RF, RNF y clasificación del sistema (Parte 1 del TP Integrador).
- /02-Arquitectura → Diagramas de información y de software.
- /03-Artefactos → Historias de usuario, casos de uso y diagramas de secuencia.
- /04-Repositorios → Este trabajo práctico (Unidad 1 – Parte 2).
- /05-Evidencias → Capturas de sprints, reuniones y avances del equipo.
- README.md → Portada del portafolio, con el índice y el estado de avance del proyecto.

Esta organización permite observar de forma ordenada y cronológica todo el trabajo realizado por el equipo durante el cuatrimestre, y queda disponible tanto para consulta de la cátedra como para la propia gestión del equipo.

5. Repositorio de Software

Plataforma seleccionada: GitHub (repositorio ya creado por el equipo).

Justificación

- Metodología de trabajo: el equipo trabaja bajo Scrum. GitHub permite modelar el Product Backlog y el Sprint Backlog mediante Issues y Projects (tablero Kanban/Scrum), asociando cada historia de usuario a una tarjeta, asignándola a un integrante y a un Milestone (Sprint).
- Características: al ser Git su base, es un sistema de control de versiones distribuido; cada integrante clona el repositorio completo y trabaja en ramas (branches) por historia de usuario, integrando los cambios mediante Pull Requests revisados por el otro integrante antes del merge.
- Tipo de repositorio: distribuido, con arquitectura equivalente a la descrita en el Gráfico 2 de la sección 3 (sincronización vía push/pull contra el repositorio remoto en GitHub).
- Clasificación: repositorio de código fuente + gestor de proyecto ágil, ya que combina control de versiones, gestión de incidencias (Issues), revisión de código (Pull Requests) y automatización (GitHub Actions/CI-CD) en una sola herramienta gratuita para uso académico.

Ejemplo de Historia de Usuario en formato de Issue

Siguiendo el formato de historias de usuario trabajado en clase (Como / Quiero / Para + Criterios de aceptación), así se cargará cada historia como Issue en GitHub:

Issue — Inscripción de alumno a actividades deportivas

Como: padre/madre

Quiero: inscribir a mi hijo en hasta dos deportes

Para: que participe de las actividades extracurriculares sin conflictos de horario

Criterios de aceptación:

1. El sistema no permite seleccionar un tercer deporte.
2. El sistema rechaza deportes con horarios superpuestos.
3. Al confirmar, la inscripción queda visible en el perfil del alumno.

Etiquetas: historia-de-usuario, alta-prioridad, sprint-2 | Milestone: Sprint 2 | Responsable: Integrante 1